

Overview of C programming language

- C programming language, often simply called C, is a powerful and popular programming language.
- It is widely used for developing a wide range of software applications, from simple programs to complex operating systems.
- C provides direct access to the computer's hardware and memory. This feature makes it highly efficient and allows programmers to have precise control over their programs.
- C is known for its simplicity and elegance, making it a great choice for beginners to start their programming journey.
- Learning C helps in understanding the fundamental concepts of programming and builds a strong foundation for learning other programming languages.
- It is a versatile language that can be used in various domains, such as system programming, game development, and embedded systems.

History and Evolution of C programming

- C was developed in the early 1970's by Dennis Ritchie at Bell Labs.
- It was initially created as a C language to write the Unix operating system.
- As Unix gained popularity, so did C. In the late 1970's, the first official specification of the C language, called "K&R", was published by Brian Kernighan and Dennis Ritchie.
- This specification became the foundation for C's widespread use and further development.
- In the 1980s, the American National Standards Institute (ANSI) standardized the C language, resulting in the creation of ANSI C or C89.
- This standardization ensured portability and compatibility of C programs across different systems.
- Over the years, C has evolved with the release of new standards like C99 and C11, introducing additional features and improvements to the language.
- C's simplicity, efficiency, and versatility have contributed to its longevity and continued relevance in modern programming. It remains one of the most widely used programming languages today, forming the basics for many other languages and frameworks.

Characteristics and Features of C.

- Firstly, C is a structured language, meaning it allows programmers to break down complex problems into smaller, more manageable parts through functions and modules.
- This makes the code easier to understand and maintain.
- C is a low-level language, providing direct access to hardware resources, such as memory and registers.
- This gives programmers precise control over their programs' performance and efficiency. Additionally, C supports a wide range of data types, allowing programmers to work with integers, floating-point numbers, characters, and more.

- It also provides powerful operators for performing mathematical and logical operations. Another important feature of C is its extensive library functions, which provide pre-written code for Common tasks, Saving time and effort.
- C is known for its efficiency, as it allows for direct manipulation of memory and offers features Like pointers, which enable advanced memory management.
- C has large Community of developers and a vast Collection of resources, making it easier.

Advantages and Disadvantages of C

Advantages of C

1. Efficiency : C is known for its efficiency in terms of memory usage and execution speed It allows for low-level manipulation of hardware , resulting in fast and optimized programmers
2. Portability: C programmes can be easily ported to different platforms and operating systems. making them highly versatile.
3. Wide Range of Applications: C is widely used in various domains, including system programming, game development, embedded systems and more. Its versatility makes it a valuable skill for programmers.
4. Powerful and Flexible : C provides a wide range of features and powerful tools, such as pointers and libraries, which allow for advanced programming techniques and flexibility.
5. Large Community and Resources: C has a vast Community of developers, making it easier to find Support, libraries, and learning resources.

Disadvantages of C

1. Steeper Learning Curve: Compared to some high-level language, C can be more challenging for beginners due to its low-level nature and complex concepts like pointers.
2. Lack of Built-in Safety Features: C does not have built-in safety features like automatic memory management or bounds checking, which means programmers need to be careful with memory allocation and error handling to avoid bugs and crashes.
3. Verbosity: C can be verbose meaning it requires writing more code to achieve certain tasks compared to high-level languages. This can make programs longer and potentially more prone to errors
4. Lack of Object-Oriented programming (OOP) Support: C is a procedural language and does not have native Support for OOP concepts like Classes & inheritance. However, it is still possible to implement OOP -like Structures in C with manual coding
5. Limited Standard Library: The Standard library. in C is relatively small compared to some other programming languages. This means that the developers may need to rely on external libraries for certain functionalities.

Structure of a C program

- A C program is Composed of various elements, including comments, preprocessor directives, function declarations and the main function. Let's break down the C program Step by step using the "Hello world" example.
- 1. Comments: Comments are used to provide explanatory notes within the code. They are not executed by the compiler but serve as useful information for programmers. In C, comments can be single-line (denoted by //) or multi-line(enclosed between /* an */).
- 2. Preprocessor Directives: Preprocessor directives begin with a hash symbol (#) and are processed by the preprocessor before Compilation. They are used to include header files, define Constants, or perform other preprocessing tasks. The most common directive is #include which allows us to include standard libraries or user-defined header files.
- 3. Main Function: Every C program must have a main function, which serves as the entry point for program's execution. The main function has a specific format: int main().It returns an integer value indicating the program's status to the operating system. Within the main function , we write the program's logic or a sequence of instructions.

The "Hello World" Example: Here's the "Hello world" program written in C :

```
#include <stdio.h>

int main()
{
    printf("hello world!");
    return 0;
}
```

Explanation:

- The # include directive includes the Standard input /output header file, which contains the declaration of the printf function.
- The main function is declared as int main() and serves as starting point of the program
- The printf function is used to print the text "Hello world !" to the Console. The return statement with the value 0 indicates that the program executed Successfully.

Basic Input/output Operations.

- In programming, input /output (I/O) Operations are essential for interacting with users and exchanging data with the computer. In this blog post, we will explore the Basic input/output operations in C programming language. Understanding how to read input from the user and display output is crucial for building interactive and useful programs. Let's dive in and learn how to perform these operations in C!
- 1. Printing Output : In C we use the printf() function to display output on the screen. It allows us to print text, numbers, and other types. The general Syntax for printf() is

```
printf ("Format String", variables);
```

- The format String contains placeholders that are replaced by the corresponding variables. For example, to print a simple message, we can use:

```
printf ("Hello, World");
```

2. Reading Input: To receive input from the user, we use the scanf() function. It allows us to read different types of data. Such as integers floating-point numbers and characters. The Syntax for Scanf() is :
 Scanf ("Format String" , variables);
 - The format String Specifies the type and format of the input, and the ampersand(&) is used to indicate the memory location where the input is stored. For example, to read an integer from the user. we can use

int num;

Scanf ("%d" &num);

Data Types and Variables.

Fundamental Datatypes in C.

In C programming language, datatypes define the nature of variables and determine the kind of data they can store. Each type has its characteristics and range of values.

1.Integer Types

- **Char** : It represents a single character and can store values from 188 to 157 (or) 0 to 355 (if unsigned)
- **int** : It is used to store integers and has a range of -32,768 to 32,767.
- **Short** : It Stores small integers with a range of -32,768 to 32,767.
- **long** : It can store larger integers with a range of -2,147,483,648 to 2,147,483,647.

2. Floating-point types:

- **float** : It represents single -precision floating point numbers with a range of approximately $+3.4e-38$ to $+3.4e+38$.
- **double** : It represents double -precision floating point numbers with a range of approximately $+1.7e-308$ to $\pm 1.7e+308$.

3. Other Data Types

- **void**: it denotes the absence of any type and is often used as a return type for functions that do not return value.
- **Bool** : It represents a Boolean value, which can be either true (1) or false(0).

Declaring and Initializing Variables.

In C programming, variables are used to store and manipulate data. Before using a variable. it needs to be declared and optionally initialized.

Declaring Variables: To declare a variable, we need to specify its datatype and a name that represents the variable. The general syntax for a variable declaration is

data_type variable_name:

float height;

Char initial;

Initializing Variables: Variable initialization is the process of assigning an initial value to a variable at the time of declaration. This ensures that the variable has a defined value before it is used in the program. This initialization can be done using the assignment operator (=). the general syntax for variable initialization is,

Data_type variable_name = initial_value;

- It is important to note that the variables must be initialized before they are used.

Type Modifiers.

- In C programming, type modifiers are used to modify the behaviour and properties of data types. They allow us to fine-tune how variables are stored and used in memory.

1. Signed and Unsigned

- The signed modifier is used with the integer data types to allow both positive and negative values. It is the default behaviour if the "signed" keyword is not explicitly used.
- The unsigned modifier is used with integer data types to allow non-negative values. It extends the range of positive values that can be stored in a variable but eliminates the ability to represent negative values.

2. Short and long

- The **"Short"** modifier is used with integer data types to allocate less memory, resulting in a smaller range of values that can be stored. It typically uses half the memory compared to the default integer type.
- The **"long"** modifier is used with integer data types to allocate more memory, allowing for a larger range of values. It typically uses double the memory compared to the default integer type.

3. Type Qualifiers

- The "Const" qualifier is used to declare a variable as constant, meaning its value **cannot** be changed once assigned.
- The "volatile" qualifier is used to indicate that a variable's value can be changed by external factors that are not within the control of the program. This is commonly used for variables that can be modified by hardware or other concurrent resources / processes.

unsigned int count = 10;

Short int temperature = -5;

long int population = 1000000 L

const float PI = 3.14159;

volatile int Sensor Value;

Constants and Literals.

- In C programming Language. Constants and literals are used to represent fixed values that do not change during the execution of a programming They provide a way to store and refer to unchanging data throughout the code.

Constants in C:

- Constants are fixed values that cannot be modified once assigned. They are created using the “Const” keyword, which ensures that their values remain Constant throughout the program. Constants are typically used to store values that are known and will not change during program execution. such mathematical Constants or Configuration values.

```
Const float PI = 3.14159;
```

```
Const int MAX_VALUE = 100;
```

Literals in C

- Literals are specific values that are directly written in the code. They represent fixed data and can be of various types. Such as integers floating-point numbers , Characters , or Strings. Literals are used to provide immediate values for calculations or to initialize variables.

```
int Score =85;
```

```
float pi value = 3.14;
```

```
char grade = "A";
```

```
Char message[] = "Hello.world!";
```

- In the above example, we assigned literals to variables. The literal values , such as 85, 3.14 , 'A' , “Hello.world!” are directly written in the code and represent fixed data.

Operators in c :

Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

1. Arithmetic Operators : Arithmetic operators are used to perform basic mathematical operations.

- Addition (+) : Adds two operands.
- Subtraction (-) : Subtracts one operand from another
- Multiplication (*) : Multiplies 2 operands.
- Division (/) : Divides one operand by another
- Modulo (%) : Calculates the remainder.

2.Relational Operators:

Used to Compare values & determine relationships --

- Equality (==): Checks if 2 operands are equal.
- Inequality (!=): Checks if 2 operands are not equal
- Greater than (>) or Greater than or equal to (>=)

Checks if operand is greater or equal to the another.

- Less than(<) or Less than or equal to (<=)

Checks if one operand is lesser or equal to the another.

3.Logical Operators:

used to perform logical operations Boolean values.

- AND (&&) : Returns true if both are true
- OR (||) : Returns true if at least one of the operands is true.
- NOT (!) : Reverses the logical state of an operand

4.Assignment Operator (=)

Used to assign a value to a variable.

```
int a = 10; // assigns 10 to a
```

```
int b;
```

```
b = a; // assigns value of a (10) to b
```

6.Increment Operator (++)

Increases a variable's value by 1.

➔ Pre-increment (++a) – increases first, then uses value.

➔ Post-increment (a++) – uses value first, then increases.

```
int a = 5;
```

```
printf("%d", ++a); // Pre-increment → a becomes 6, then prints 6
```

```
printf("%d", a++); // Post-increment → prints 6, then a becomes 7
```

```
printf("%d", a); // Now a is 7
```

7. Decrement Operator (--)

Decreases a variable's value by 1.

➔ Pre-decrement (--a) – decreases first, then uses value.

➔ Post-decrement (a--) – uses value first, then decreases.

5. Control Flow Statements in C

Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

1. if - Statement:

The "if" Statement allows you to execute a block of code if a specified condition is true. If the condition is false, the code block is Skipped. The general syntax is,

```
if(condition)
{
// code to be executed if the condition true .
}

int age =25;
if (age >= 18)
{
printf("You are eligible to vote. ");
}
```

2. if – else Statement:

The "if-else" Statement allows you to execute one block of code if a condition is true and another block of code if the condition is false. The general Syntax :

```
if (condition)
{
}

Else
{
}

int marks = 85;
if (marks >=80)
{
printf ("you passed the exam")
}

Else
{
printf("you failed the exam");
}
```



```
}
```

3. Nested if-else Statement:

The nested if-else statement allows you to have multiple levels of if-else statements within each other. This allows for more complex decision-making Scenarios. The general syntax is:

```
if (condition1)
{
    if (condition2)
    {
    } else
    {
    }
} else {
    if (condition3) {
    }
}
```

```

1
2 //nested if
3
4 #include <stdio.h>
5 #include <conio.h>
6
7 int main() {
8     int age;
9     printf("Enter your age: ");
10    scanf("%d", &age);
11
12    if (age >= 18) {
13        printf("You are an adult.\n");
14        if (age >= 65) {
15            printf("You are a senior citizen.\n");
16        }
17    } else {
18        printf("You are a minor.\n");
19    }
20
21    return 0;
22 }
23
24

```

```

Enter your age: 15
You are a minor.

```

5. Switch – statement

- switch is a decision-making statement in C. It is used when we want to choose one option out of many based on the value of an expression. It works like multiple if-else statements, but is more structured and readable.

```
main.c
1  #include <stdio.h>
2  #include <conio.h>
3
4  int main() {
5      int num;
6
7      printf(" week day number : ");
8      scanf("%d", &num);
9
10     switch(num) {
11         case 1:
12             printf("\n Monday");
13             break;
14         case 2:
15             printf("\n Tuesday");
16             break;
17         case 3:
18             printf("\n Wednesday");
19             break;
20         case 4:
21             printf("\n Thursday");
22             break;
23         case 5:
24             printf("\n Friday");
25             break;
26         case 6:
27             printf(" \n Saturday");
28             break;
29         case 7:
30             printf(" \n Sunday");
31             break;
32         default:
33             printf(" \n wrong day number ");
34     }
35
36     return 0;
37 }
38
```

week day number : 5

Friday

...Program finished with exit code 0
Press ENTER to exit console.

6.Looping in c

Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

1.While loop :

- The condition is checked **before** running the loop.
- If condition is false at the start, the loop won't run at all. The general syntax is

While(condition)

{ // code }

2.For loop :

- This is Entry control loop.

initialization → set starting value (e.g., int i=1;)

condition → loop continues while true (e.g., i<=10)

update → change variable each iteration (e.g., i++). The general syntax is

For(initialization; condition; update) {

 // code to execute

}

3.Do while :

- This is exit control loop. A **do...while loop** in C language is used when you want the loop body to execute **at least once**. the general syntax is

Do

{

 // code to execute

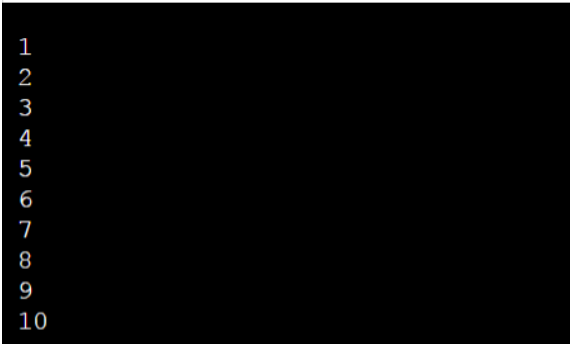
}while (condition)

Lab exercise

Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

for loop

```
main.c
1 #include<stdio.h>
2 #include<conio.h>
3
4 int main()
5 {
6     int i;
7     for (i=1; i<=10;i++)
8         printf( "\n %d",i );
9     return 0;
10 }
```



while loop

```
main.c
1 #include<stdio.h>
2 #include<conio.h>
3
4
5 int main() {
6     int i = 1;
7     while (i <= 10) {
8         printf("\n %d", i);
9         i++;
10    }
11    return 0;
12 }
13
14
```



do while loop

```
main.c
1 #include<stdio.h>
2 #include<conio.h>
3
4 int main() {
5     int i = 1;
6     do {
7         printf("\n %d", i);
8         i++;
9     } while (i <= 10);
10    return 0;
11 }
12
13
```



7. Loop Control Statements

1. break Statement :

- Used to terminate the loop immediately.
- Control transfers to the first statement after the loop.
- Commonly used in switch and loops.

```
main.c
1  #include <stdio.h>
2  #include <conio.h>
3
4  int main()
5  {
6      int i;
7      for (i = 1; i <= 10; i++)
8      {
9          if (i == 5)
10         {
11             break;
12         }
13         printf("%d ", i);
14     }
15     return 0;
16 }
17
```

1 2 3 4

...Program finished with exit code 0
Press ENTER to exit console.

2. continue Statement

- Used to **skip the current iteration** of the loop.
- Control moves to the **next iteration** of the loop.

```
main.c
1  #include <stdio.h>
2  #include <conio.h>
3
4  int main()
5  {
6      int i;
7      for (i = 1; i <= 5; i++) {
8          if (i == 3) {
9              continue;
10         }
11         printf("%d ", i);
12     }
13     return 0;
14 }
15
16
```

1 2 4 5

...Program finished with exit code 0
Press ENTER to exit console.

3.Go to statement

- The `goto` statement in C is a control flow tool that allows you to jump to another part of the same function using a label.
- It's like teleporting within your code—but with great power comes great responsibility!
- Breaking out of deeply nested loops.
- Avoiding repetitive code in specific scenarios.

```
main.c
1  #include <stdio.h>
2  #include <conio.h>
3
4  int main() {
5      int num = 5;
6
7      if (num == 5)
8          goto skip;
9
10     printf(" \n This won't be printed.");
11
12     skip:
13     printf(" \n Jumped to label 'skip'.");
14
15     return 0;
16 }
```

Jumped to label 'skip'.

8. Functions in C

What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

- In C programming, functions are blocks of code designed to perform a specific task.
- They help break down complex problems into smaller, manageable parts, promote code reuse, and improve readability.

Types of Functions

1. **Library functions** – Already defined in C libraries (e.g., `printf()`, `scanf()`, `sqrt()`). These functions help perform common tasks like input/output operations, mathematical calculations, string manipulations, etc.

2. **User-defined functions** – Functions created by the programmer. A user-defined function is a function created by the programmer to perform a specific task. These functions improve modularity and make programs easier to manage.

3. User-Defined Functions in C (Summary Table)

Type	Arguments	Return Value	Syntax Example	Explanation
1. No arguments, No return value	✗	✗	<code>void funcName(void);</code>	Function neither takes input nor returns output.
2. With arguments, No return value	✓	✗	<code>void funcName(int a, int b);</code>	Function takes input but does not return a result.
3. No arguments, Returns value	✗	✓	<code>int funcName(void);</code>	Function does not take input but returns a result.
4. With arguments, Returns value	✓	✓	<code>int funcName(int a, int b);</code>	Function takes input and also returns a result.

Function Declaration in C

A function declaration (also called a *prototype*) tells the compiler:

- Function name
- Function return type
- Number and types of parameters

- It is written before main() so that the compiler knows about the function before its actual use.
- `return_type function_name(parameter_list);`

Function definition

- A function definition provides the actual body of the function — i.e., the block of code that performs the task.

```
int add(int a, int b)
```

```
{
```

```
return a + b; }
```

EXAMPLE

```
main.c
1  #include <stdio.h>
2  #include <conio.h>
3
4
5  // Function Declaration
6  int sum(int, int);
7
8  int main() {
9      int result;
10     result = sum(6, 7); // Function Call
11     printf("Product = %d", result);
12     return 0;
13 }
14
15 // Function Definition
16 int sum(int a, int b) { // function body
17     return a * b; // returns product
18 }
19
```

Product = 42

...Program finished with exit code 0
Press ENTER to exit console.

9.Arrays in C

Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

Introduction to Arrays in C

- An array is a collection of elements of the same data type, stored in contiguous (continuous) memory locations, and accessed using a single name with an index.
- They provide an efficient way to manage and manipulate a group of related data.
- They play a crucial role in tasks storing a list of names, recording scores ,or managing large sets of data efficiently.
- With array you can work with multiple values, conveniently and streamline your code.

One dimensional array and multi-dimensional array

One dimensional array :-

- One dimensional array is a liner collection of elements of the same data type. It is a like a simple list of values stored in a single row.
- Each element in the array is accessed using its index, which starts from 0.
- One dimensional arrays are commonly used to store a sequence of values , such as a list of numbers , names or grades.
- They provide a convenient way to organize and access data In a linear manner.
- Ex. `Int a[5]; // declaration`
`a[5]={1,5,,4,15,20} // intialization .`

Multi-dimentional array :-

- a multi dimensional array is an extension of the one-dimentional array,but it has two or more dimensions.
- It is like a table or grid of elements arranged in rows and columns.
- Each elements in a multi dimensional array is accessed using multiple indices.
- Multi-dimentional arrays are commonly used to represent matrices,tables or other structured data.
- Ex. `Int a[2][2]`
`a[2][2]=[[1,2] , [3][4]];`
- Arrays , both one-dimentional and multi-dimentional are powerful tools in c programming.
- The number of dimensions in an array determines the complexity of organizing and accessing data.
Ex. `Int a[2][2];`

```
a[2][2]=[[1,2],[3,4]];
```

- Arrays both one-dimensional and multi-dimensional are powerful tools in C language.
- The number of dimensions in an array determines the complexity of organizing and accessing data.

Array initialization and manipulation

- Arrays are a fundamental data structure in C that allow you to store and manipulate a collection of data.
- Now, we will discuss initializing arrays, modifying array elements, and performing common operations on arrays.

Array initialization

- To initialize an array, you assign values to its elements during declaration or after declaration using assignments, statements. During declaration you can provide the initial values enclosed in curly braces {} within the square brackets [].

Ex. `int number [5]={1,2,3,4,5}`

Accessing array elements :-

- Array elements are accessed using their indices, starting from 0 to retrieve the value of an element, you use the array name followed by the index in square brackets.

Ex. `int third number = numbers[2];`

- In the above example we access the third element of the "number" array and assign it to the variable "third number".

Modifying Array Elements

- Array elements can be modified by assigning new values to them using their indices.

Ex:

`numbers [1]=10;`

- In the above example, we change the value of the second element of the "numbers" array to 10.

Common Array Operations :-

- Arrays support various operations such as finding the length of an array, traversing array elements using loops, and performing calculations on array elements.

Ex :-

`int length = sizeof (numbers)/sizeof (numbers[0]);`

```

for(int i = 0; i < length; i++)
{
printf ("%d", numbers[i]);
}

```

- In the above example, “length” is getting the size of the “numbers” array. we then use a for-loop to traverse the array and print each element.

10. Introduction to pointers.

Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

- Pointers are powerful and fundamental concepts used to work with memory addresses . A pointer is a variable that holds the memory address of another variable.
- Pointers are declared using asterisk(*) symbol of another the variable name.

Ex

```
int *ptr; // declares a pointer to an integer
```

- Pointers store memory addresses, which represent the Location where a variable is stored in the Computer's memory.
- The address- of operator (&) is used to obtain the memory address of a variable.

Ex

```
int num =5;
```

```
int *ptr = & num ;
```

- Assigns the memory address of ‘num’ to the pointer ‘ptr’.
- pointers allow us to indirectly access and manipulate data by dereferencing them using the dereference operator (*).

Ex

```
*ptr =10;
```

*10 is assigned memory location of ptr.

- pointers are used for dynamic memory allocation, such as creating arrays and Structures at runtime.
- it is important to initialize pointers before using them to avoid accessing uninitialized or invalid memory addresses.
- Pointers provide a way to optimize code and efficiently manage memory resources in C programs.

Pointer Arithmetic.

- In C programming, pointer arithmetic refers to performing mathematical operations on pointers. pointer arithmetic is a powerful tool that enables efficient manipulation of data structures and arrays.
- pointer arithmetic is performed on pointers that point to elements of an array or allocated memory.
- when adding or subtracting an integer value to / from a pointer, the result is adjusted based on the size of the data type the pointer points to. pointer arithmetic is closely related to array indexing . array indexing is a form of pointer arithmetic.

Ex:

Arr[2] is equivalent to *(arr+2).

- Pointer arithmetic can be used to iterate over an array or access elements at specific memory locations. Accessing memory beyond the allocated space can result in undefined behaviour and potential Crashes. Pointers can be incremented or decremented by any integer value.

Ex

“Ptr ++ ” moves the pointer to next memory location.

“ptr--” moves it to the previous Location.

- Pointer arithmetic can be combined with other operators, such as assignment and conditional operators, to perform more complex operations.
- Understanding and utilizing pointer arithmetic can greatly enhance your ability to work with data structures and efficiently manage memory in C programs.

11. Introduction to Strings

Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr().

Provide examples of when these functions are useful.

- In C programming language, Strings are used to store and manipulate sequence of characters. Strings are essential for working with textual data like names, Sequences, or any other collection of characters.
- In C, Strings are represented as arrays of characters terminated by a null Character ‘\0’ : The null character indicates the end of the string.

Ex

Chare greeting[10] = "Hello; //declaration & initialization ;

- We declared string “greeting” and initialize with the value “hello”. The size of the array should be large enough to accomodate the characters in the String, including the null character.
- Strings are widely used in programming, particularly when dealing with textual data.
- They provide a convinient and efficient way to work with words, sentences, and other character sequences.

String Manipulation Functions

- In C programming language. String manipulation functions are used to perform various operations on string.
- These functions provide convenient ways to manipulate and process textual data.

1. Strlen

Size + strlen (const char * str)

- The Strlen() function calculates the length of a String by counting the number of characters until it reaches the null character '\0'.
- It returns the length of the string as a size_t (unsigned integer) value.

2. Strcpy ():

Char*strcpy(char*destination,const char*source)

- The strcpy() function is used to copy the Contents of one string to another. It takes destination string and source string as arguments.
- The function copies the characters from the source string to the destination String, including the null character "\0".
- It returns pointer to the destination

3.Strcat():

Char*strcat (char*destination, const char*source)

- The strcat() function concatenates (joins) two Strings together.
- It appends the Characters from the source string to the end of the destination string, overwriting the null character of the destination string.
- The function returns a pointer to the destination String.

4.Strcmp():

int strcmp (const Char*str1,const char*str2)

- The strcmp() function compares two strings lexico graphically.
- It returns an integer value that indicates the relationship between the strings.
- values : 0 if the strings are equal.
 1 if first string is less than 2nd String
 +1 if 1st string is greater than 2nd

5.Strstr ():

```
Char*strstr(const char*haystack,const char*needle)
```

- The strstr() function searches for the first occurrence of a substring within a String.
- It takes haystack string (larger string to search within) and the needle string (the substring search for).
- It returns a pointer to the first occurrence of the needle string within the haystack String.
- It returns null pointer, if the substring is not found.

12. Structures in C

Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

Defining Structures.

- In C programming, Structures provide a way to Combine different data types into a single entity. A structure is a user-defined data type that allows you to group related variables together
- It helps in organizing and managing complex data structures. To define a structure, you specify its name and the variables it contains.
- Each variable within the structure is called a member. Ex

```
struct person
{
    Char name[50];
    Int age ;
    Float height ;
};
```

- In this example, we defined a structure called 'person' with 3 members : 'name' an array of characters to store the person's name 'age' Can integer to store the person's age; and 'height' (a float to store the person's height)

Accessing Structure Members.

- Once you have defined a structure, you can Create Variables of that structure type and access its members using the dot (.) operator.
- The dot operator allows you to access individual members of a structure.
- Example for creating a variable of the 'person' Structure and accessing its members:

```
Struct person p ;
Strcpy(p.name, john);
p.age = 25 ;
p.height = 1.75 ;
```

- In this example, we created a variable 'p' of type 'person'. we then assigned values to its members using the dot operator.

- The 'strcpy' function is used to copy the String "John" into the "name" member.

Nested Structures.

- In C. you can also define structures within Structures , known as nested structures.
- This allows you to Create more complex data structures by Combing multiple Structures.

Ex

Struct date

```
{
  Int day;
  Int month ;
  Int year ;
};
```

Struct person

```
{
  char name [50];
  struct date birthdate;
};
```

- In this example, we defined a structure called 'Date' to represent a date with day month, and year.
- Then, we defined another structure called 'person' , which includes the 'name' member and a nested Structure 'birth date' of type 'Date'.

Structure and Functions.

- Structures can be passed as arguments to functions and returned from functions, allowing you to operate on complex data structure.

```
struct person
{
  char name [50];
  int age
};
void print person(struct person p)
{
  printf("\n Name : %s",p.name);
  printf("\n Age : %d",p.age);
}
```

- In this example, we defined a structure 'person' and function 'print person' that takes a 'Person' Structure as an argument.
- The function prints the name and age of the person.

13 File handling in c.

Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

File Concepts and Operations

- **File** : A file is a named collection of related data Storage medium, Such as hard-drive or solid-state drive. It can be a text file or a binary file.

- **File Operations** : C provides several functions for file operations, including opening, reading ,writing closing and manipulating data or files. These functions are part of the standard library, Such as 'fopen' , 'fread' , 'fwrite' , 'fclose' . etc.
- **File Pointers** : When working with files , you use a file pointer, which is a special variable that represents the file being accessed.
- The file pointer keeps track of the current position in the file during read and write operations.
- File pointers allows you to perform reading writing, opening, closing files operations.
- A file is a named collection of data stored on a storage device such as a hard disk.

Opening and closing Files.

- Before you can perform any operations on a file, you need to open it.
- To open a file, you use the 'fopen()' function, which takes two parameters : the file name and the mode in which you want to open the file

FILE *filePointer:

```
filePointer = fopen ('example.txt', 'r');
```

- In this example, we declared a 'filepointer' and used the 'fopen()' function to open the file named 'example.txt' in read mode ("r").
- The 'fopen()' function returns a pointer to the apened file, which we assigned to the file pointer.
- After performing file operations, it's important to close the file using the 'fclose()' function. Closing the file ensures that any changes made to the file are saved and frees up system resources

Ex : fclose (file pointer);

- In this example, we used the 'fclose()' function to close the file pointed to by 'filepointer'

Reading and writing Files.

- Once you have opened file, you can read data from it or write data to it using different file handling functions .
- To read data from a file, you can use functions like 'fscanf' or 'fgets()'. These functions allow you to read data from the file and store it in variables.

Ex : reading data from the file using "fgets()"

```
Char buffer [100];
```

```
fgets ( buffer , 100, file pointer);
```

- In this example, we used the "fgets()" function to read a line of text from the file pointed to by 'file pointer' and Store it in the character array 'buffer' .
- To write data to a file, you can use functions like 'fprintf()' or 'fputs'.
- These functions allow you to write data to the file using formatted or non-formatted output.

Ex - writing data to the file using 'fprintf' .

```
int number = 48;
```

```
fprintf (file pointer, "The answer %d",number);
```

- In the above example, we used the 'fprintf()' function to write the formatted output "the answer 48" to the file pointed to by file pointer.

Error Handling in File Operations.

- When working with files, it's important to handle errors that may occur during file Operations.
- File operations can fail due to various reasons, Such as the file not existing, insufficient permissions, or disk errors.
- To handle errors in file operations, you can Check the return value of the file handling functions.
- Most file handling functions return a value that indicates success or failure.
- you can use this value to determine if the operation was successful or if an error occurred.

Ex:

```
error handling when opening a file  
FILE *file Pointer;  
filePointer = fopen ("example.txt", "r");  
if(filePointer ==NULL)  
{  
    printf("error opening the file. \n");  
}
```

- In this example. we checked if the file pointer returned by "fopen()" is 'NULL' Which indicates that an error occurred during file Opening.